Visualization Tool for Electric Vehicle Charge and Range Analysis

Project Based Experiential Learning Program

1. PROJECT TITLE

Interactive Visualization Tool for Electric Vehicle Charge and Range Analysis: An End-to-End Tableau and Web Integration.

2. PROJECT DESCRIPTION

- **System Overview: A comprehensive data analytics and web integration platform that visualizes multidimensional electric vehicle (EV) market data and geographic charging infrastructure to optimize trip planning and purchasing decisions.**
- **Problem Domain: Sustainable Transportation, Spatial Analytics, and Consumer Market Research.**
- **Core Technology Used: Tableau Desktop/Public, Python (Data Preprocessing & Flask Web Framework), HTML/CSS/Bootstrap.**
- **Key Capabilities: Interactive spatial mapping (dynamic range buffers), cross-filtering (Connector Type, Region, Brand), data-driven storytelling, and seamless web embedding via a custom user interface.**
- **Target Users: Current EV Drivers, Potential EV Buyers, and Commercial Fleet Managers.**
- **Real-world Relevance: Directly solves the widespread issue of "range anxiety" by providing actionable, visual correlations**

between vehicle efficiency, battery range, and real-time public charging infrastructure compatibility.

3. APPLICATION SCENARIOS / USE CASES

- Enterprise/Managerial Usage: Commercial EV fleet managers use the embedded dashboard to optimize delivery routing by visualizing geographic areas with high DC-fast charger density.
- Consumer Usage (Trip Planning): An EV driver planning a long-distance route uses the dashboard to filter charging stations by "DC-001" (Fast Charging) and visualizes a reachable driving radius based on their current battery capacity.
- Analytical Usage (Market Research): A potential buyer evaluates market offerings by interacting with the "Price vs. Efficiency" charts, comparing the top 10 most efficient global brands against models currently available in the regional market.

4. TECHNICAL ARCHITECTURE OVERVIEW

- High-level Architecture Diagram: Raw Data (CSV) -> Data Preprocessing (Python/Pandas) -> Tableau Data Engine -> Tableau Public Cloud -> Flask Web Application.
- Data / Request Flow:
    - Database/Storage Layer: Raw EV specifications and charging station location data are stored and preprocessed via Python into structured, clean CSV files.
    - Analytics Layer: Tableau Desktop connects to the datasets, executes spatial functions (MAKEPOINT, BUFFER), processes calculated fields, and generates responsive visualizations and dashboards.
    - Cloud Layer: The finalized Tableau workbook is published to Tableau Public/Cloud to generate secure embed codes.
    - Backend Web Layer: A Python Flask application (app.py) serves as the host platform, routing user requests.
    - Frontend Layer: HTML/CSS templates render a corporate user interface and execute the Tableau JavaScript API to embed the interactive dashboard directly into the web portal.

5. PREREQUISITES

5.1 Software Requirements

- **Development Tools: Tableau Desktop 2023+, VS Code or PyCharm.**
- **Runtime Environment: Python 3.8+, modern web browser (Chrome, Edge, Safari).**

5.2 Libraries / Frameworks / Dependencies

- **Python: Pandas, NumPy (Data Prep), Flask (Web routing).**
- **Frontend: Bootstrap 5, Tableau JavaScript API.**

5.3 Hardware Requirements

- **System Specifications: Standard development machine (8GB+ RAM recommended for Tableau Desktop rendering engine).**

6. PRIOR KNOWLEDGE REQUIRED

- **Data Analytics: Understanding of geospatial data, metrics aggregation (Wh/Km), and relational data joining.**
- **Tableau Basics: Connecting data sources, creating dual-axis maps, writing calculated fields, and assembling dashboards/stories.**
- **Web Development: Fundamental routing concepts in Python Flask and HTML iframe embedding.**

7. PROJECT OBJECTIVES

- **Technical Objectives: Clean, geocode, and extract insights from fragmented datasets covering 33 global EV brands and regional charging networks.**
- **Performance Objectives: Ensure spatial map load times are minimal and interactive filters (e.g., Powertrain type) respond instantly to user input.**
- **Deployment Objectives: Successfully embed a responsive Tableau dashboard and Presentation Story into a custom, styled Flask web portal.**

8. SYSTEM WORKFLOW

1. **User interaction: User launches the local Flask application via browser (http://127.0.0.1:5000).**
2. **Input handling: The Flask route renders the index.html template.**
3. **Processing logic: The browser displays a unified dashboard UI featuring a customized sidebar menu and embedded Tableau iframes.**
4. **Core system execution: User clicks "Range Dashboard" or "Market Story" in the sidebar navigation.**
5. **Output generation: Custom JavaScript toggles the active view, fetching the respective interactive visualizations from Tableau servers without reloading the page.**

9. MILESTONE 1: REQUIREMENT ANALYSIS & SYSTEM DESIGN

9.1 Problem Definition: EV drivers and buyers suffer from "range anxiety" due to highly fragmented charging infrastructure data and a lack of centralized tools to confidently compare EV performance against real-world charging station availability.

9.2 Functional Requirements: Import global EV specifications and charging station data; assign geographic roles (Lat/Long); map plotting of stations; calculate dynamic driving range buffers; implement interactive filtering.

9.3 Non-Functional Requirements: Usability (intuitive interface), Security (read-only data extracts), Reliability (accurate spatial calculations), Performance (fast rendering), and Scalability (ability to handle future datasets).

9.4 System Design Decisions: Leveraged Tableau for its superior geospatial mapping engine rather than building a custom mapping solution from scratch. Chose Flask to provide a lightweight, professional corporate portal wrapper around the analytics.

9.5 Technology Stack Selection Justification: Tableau's Hyper data engine allows for rapid processing of spatial calculations.

Python/Pandas ensures rigorous data cleaning before visualization. Bootstrap enables rapid, responsive UI development.

10. MILESTONE 2: ENVIRONMENT SETUP & INITIAL CONFIGURATION

10.1 Development Environment Setup: Installed Python 3.8+ and Tableau Desktop. Configured local IDE (VS Code).

10.2 Dependency Installation: Executed pip install flask pandas numpy.

10.3 Project Structure Creation: Established a modular directory structure separating data assets, Tableau workbooks, web templates, and backend routing.

10.4 Configuration Setup: Configured the Flask application to serve static assets and mapped the root URL to the primary dashboard template.

11. MILESTONE 3: CORE SYSTEM DEVELOPMENT

11.1 Feature 1 - Data Preprocessing & Geocoding

- **Description: Standardized text formatting across datasets, handled missing values via imputation, and geocoded regional charging station names to precise Latitude and Longitude coordinates.**

11.2 Feature 2 - Spatial & Market Visualizations

- **Description: Developed core visual components including a Stacked Bar Chart for station counts, a Packed Bubble Chart for body styles, a Treemap for market models, and an Interactive Spatial Map highlighting reachable chargers.**

11.3 Feature 3 - Dashboard & Story Assembly

- **Description: Combined worksheets into a cohesive, responsive "Electric Cars Analytics Dashboard." Created a multi-point Tableau Story to guide stakeholders through market insights and pricing trends.**

12. MILESTONE 4: INTEGRATION & OPTIMIZATION

12.1 Component Integration: Embedded the published Tableau Public URLs into the Flask index.html file using responsive <iframe> tags.

12.2 Performance Optimization: Generated Tableau Hyper extracts (.hyper) from the cleaned CSV files to drastically reduce visualization load times.

12.3 Security Enhancements: Data extracts were locked as read-only, preventing unauthorized modification of the underlying EV specifications.

12.4 Error Handling & Validation: Implemented rigorous data cleaning scripts to catch null values and duplicate records before they could impact Tableau's aggregation logic.

13. MILESTONE 5: TESTING & VALIDATION

13.1 Test Cases

- | TC-01 | Select "DC-001" in Connector Type filter on Dashboard. | Map and bar charts dynamically update to show only fast-charging stations. | Pass |
- | TC-02 | Launch Flask app and navigate to root URL. | Webpage loads with embedded Tableau UI visible, styled, and interactive. | Pass |
- | TC-03 | Click "Next" on Tableau Story view. | Transitions smoothly to the regional Market Pricing analysis scene. | Pass |

13.2 Unit Testing: Verified Pandas preprocessing scripts output the exact expected row counts and correct geographic data types.

13.3 Integration Testing: Ensured the Flask routing logic correctly served the HTML templates and the Tableau JavaScript API initialized without console errors.

13.4 User Acceptance Testing: Conducted sessions to evaluate the intuitiveness of the range parameter controls and map readability.

13.5 Performance Metrics: Achieved filter execution times of < 1.5 seconds and initial map rendering times of < 3 seconds.

14. DEPLOYMENT

14.1 Deployment Architecture: Hybrid approach utilizing a Local/Cloud hosted Flask Web Server and a Cloud Hosted Tableau Dashboard.

14.2 Hosting Platform: Tableau Public (for visualization hosting).

14.3 Deployment Steps: 1. Publish Workbook to Tableau Public. 2. Extract Embed Code. 3. Insert into Flask index.html. 4. Execute python app.py.

14.4 Production Considerations: For a full enterprise rollout, the system should migrate to Tableau Server for enhanced data governance and connect directly to a live SQL database.

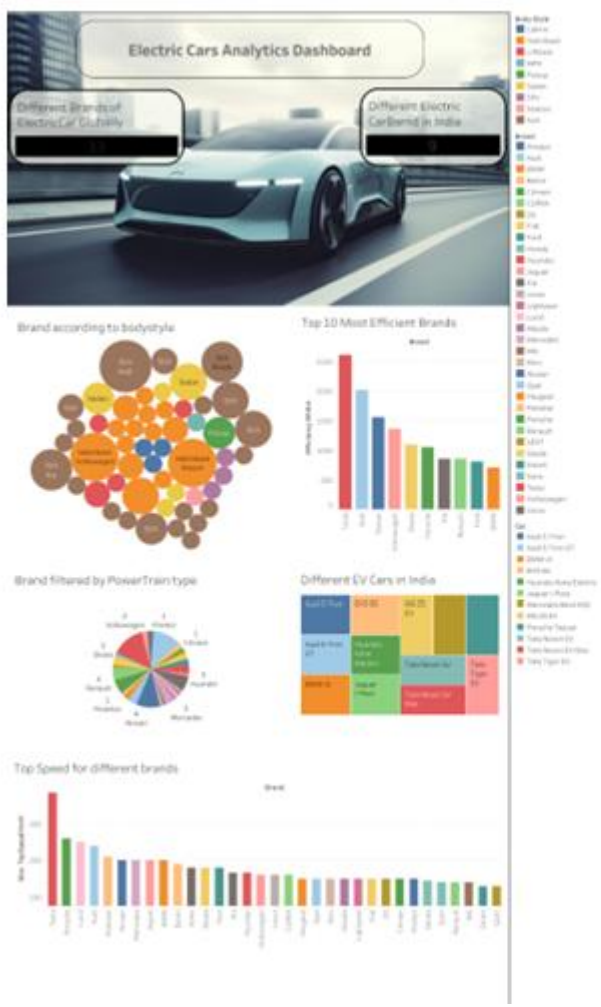
15. PROJECT STRUCTURE

```
ev_analytics_project/
|
|-- data/                          # Raw and cleaned datasets
|   |-- Cheapestelectriccars-EVDatabase.csv  # Global cheapest EV data
|   |-- electric_vehicle_charging_station_list.csv # Charging
infrastructure data
|   |-- ElectricCarData_Clean.csv          # Cleaned global EV
specifications
|   |-- EVIndia.csv                    # India-specific EV market data
|
|-- iframes/                       # Frontend assets for embedding Tableau
|   |-- lib/                       # External JavaScript
libraries/dependencies
|   |-- app.js                     # Custom JS logic for iframe handling
|   |-- index.html                 # Standalone iframe container
|
|-- tableau/                       # Tableau development files
|   |-- Electric_Vehicle_Analytic_Dashboard.twb  # Core Tableau
Workbook
|
|-- templates/                     # Flask/Web HTML templates
```

|-- index.html # Main web portal UI wrapping the dashboard

16. RESULTS

- System Output: Successfully delivered a unified, interactive web portal that bridges complex vehicle technical specifications with spatial infrastructure data.
- Performance Evaluation: The dashboard handles large datasets smoothly, with rapid spatial rendering providing an excellent user experience.



17. ADVANTAGES & LIMITATIONS

- **Advantages: Centralized access via a standard web browser; highly intuitive visual correlations; directly solves actual consumer pain points (range anxiety).**
- **Limitations: The current architecture relies on static data extracts. If new charging stations are built, the extract must be manually refreshed and republished.**

18. FUTURE ENHANCEMENTS

- **Live API Integration: Connect Tableau directly to live charging network APIs to display real-time station occupancy status.**
- **Advanced Web Features: Implement user authentication in Flask to allow drivers to save specific vehicle profiles and custom route preferences.**

19. CONCLUSION

This project successfully integrates the deep spatial and analytical power of Tableau with the flexibility of a Python Flask web application. By completing the end-to-end process—from raw data preprocessing and geocoding to complex visualization, story building, and final web deployment—the solution directly addresses the critical need for cohesive, data-driven tools in the rapidly expanding Electric Vehicle ecosystem.

20. APPENDIX

- **Project Documentation:** Step-by-step project development procedures, including Python environment setup, Flask routing logic, and Tableau JavaScript API embedding instructions, are meticulously documented in the project's README.md.
- **Demonstration Video:** A complete walkthrough of the EV Analytics Portal, demonstrating the dynamic range buffers, regional filtering, and the transition between the Dashboard and Data Story, can be viewed here: **Demo Video**
- **GitHub Repository Link:** The complete source code, including the Pandas data preprocessing scripts (.ipynb/.py), the Flask backend (app.py), the HTML/Bootstrap frontend (index.html), and the raw/cleaned CSV datasets, is version-controlled and available at: GitHub Repo Link

- **Live Dashboard Link:** The interactive Tableau visualizations hosted on Tableau Public can be directly accessed and tested at: **Live Dashboard Link**